Adventist University of Central Africa
P.O. Box 2461 Kigali, Rwanda | www.auca.ac.rw | info@auca.ac.rw

SOFTWARE SECURITY

MINI PROJECT REPORT

LAB 2: BUFFER OVERFLOW EXPLOITATION

Student Name: UMUTONI Hussina

Student ID: 28222

Course: Software Security

Submission Date: July 17th 2026

Lecturer: NKAKA Yves

1. Introduction

A buffer overflow is a type of vulnerability that occurs when a program receives more input data than its memory buffer is designed to hold. The excess data overflows into adjacent memory areas, corrupting stored values including the instruction pointer (EIP), which tells the processor what to execute next. An attacker who controls the EIP register can redirect the program to execute their own code instead of the intended instructions.

This lab demonstrates how a buffer overflow vulnerability can be discovered and exploited in a controlled environment using two virtual machines. The target machine runs Windows 10 with Vulnserver, a program that is intentionally designed to be vulnerable. The attacking machine runs Kali Linux. Immunity Debugger is used on the Windows 10 machine to observe memory and program behavior during the attack.

2. Lab Setup

The lab was conducted using two virtual machines running side by side inside Oracle VirtualBox on the same host computer. Both machines were connected to the same virtual network so that they could communicate with each other.

Role	Machine	IP Address
Attacker	Kali Linux	192.168.50.192
Victim	Windows 10	192.168.50.30

Tools used in this lab:

Vulnserver: A deliberately vulnerable server application running on Windows 10. It listens for connections on port 9999 and supports several commands, including the TRUN command which is vulnerable to buffer overflow.

Immunity Debugger: A debugging application installed on Windows 10. It allows the program memory, registers, and execution state to be observed in real time while the application is running.

Mona.py: A plugin for Immunity Debugger used to automate memory searches such as finding JMP ESP addresses.

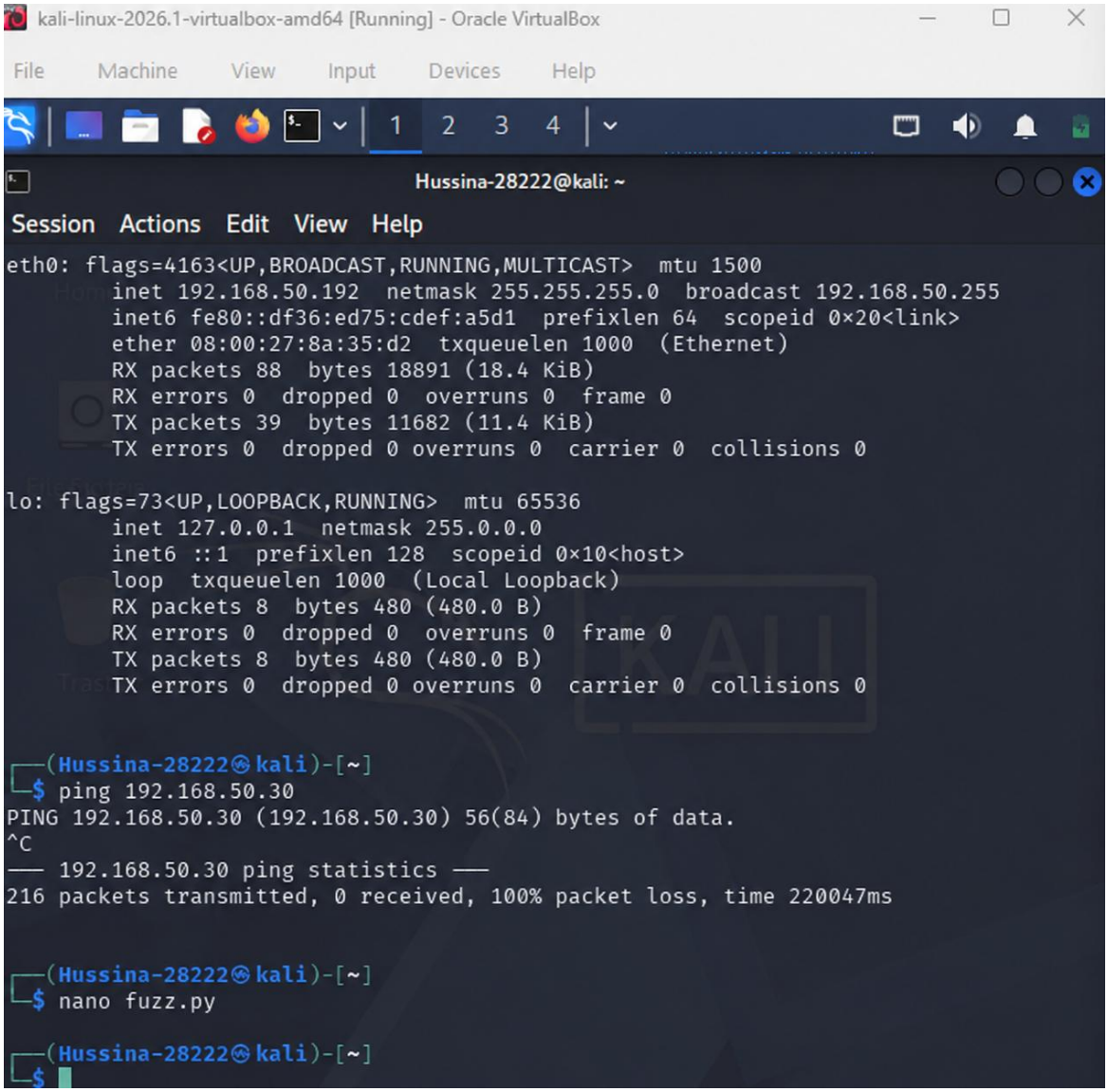
Netcat (nc): A command-line networking tool used on Kali Linux to test connections and to receive incoming reverse shells.

Python 3: Used to write and run the fuzzing script and the final exploit on Kali Linux.

3. Steps Performed

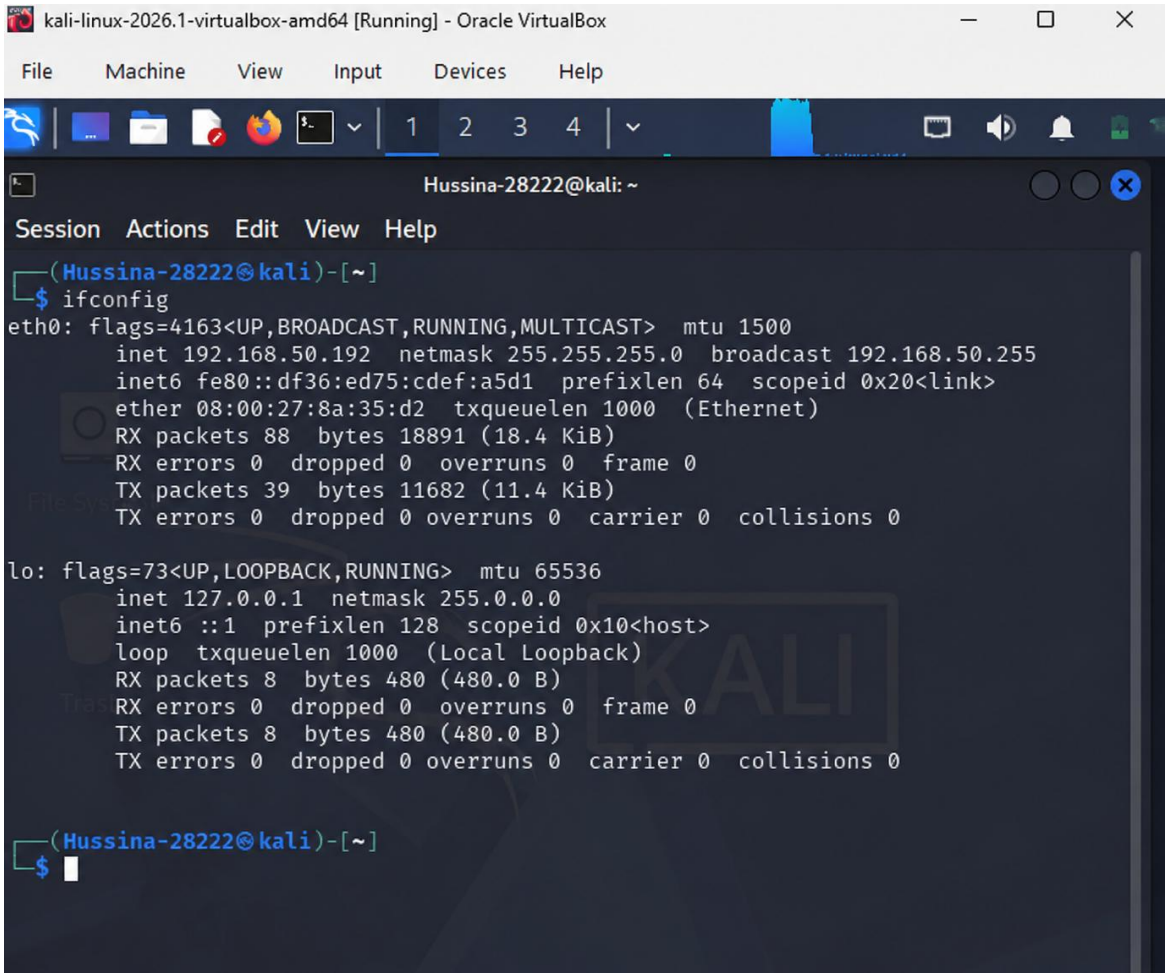
3.1 Verifying the IP Addresses of Both Machines

Before starting any attack, it was necessary to confirm the IP addresses of both machines so that the fuzzing script could be configured with the correct target. On the Kali Linux machine, the command "ifconfig" was run in the terminal. The output showed that the eth0 network interface had the IPv4 address 192.168.50.192.



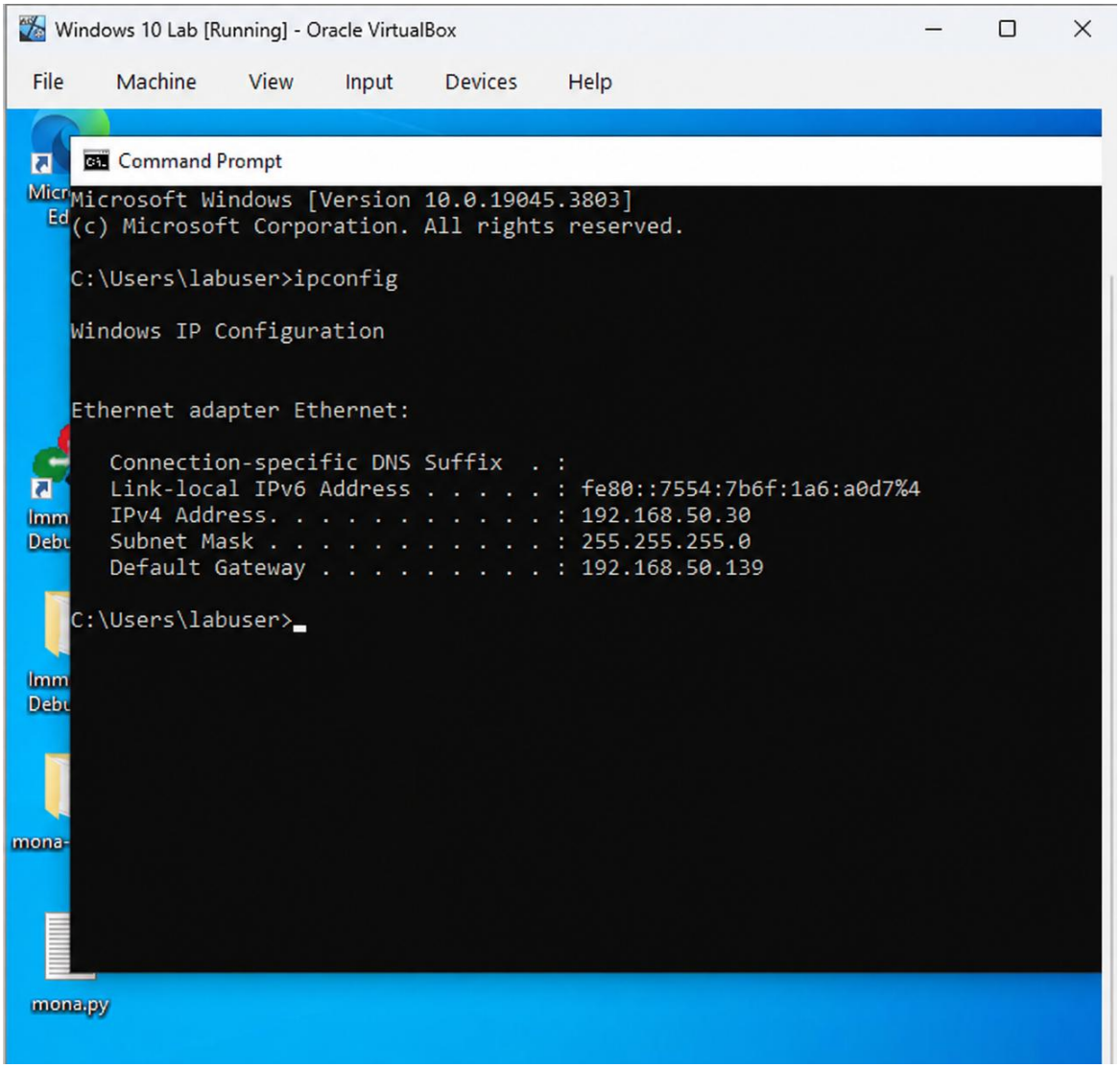
Screenshot 1: ifconfig output on Kali Linux showing eth0 IP address as 192.168.50.192.

The same ifconfig command was run a second time to confirm the address had not changed before proceeding.



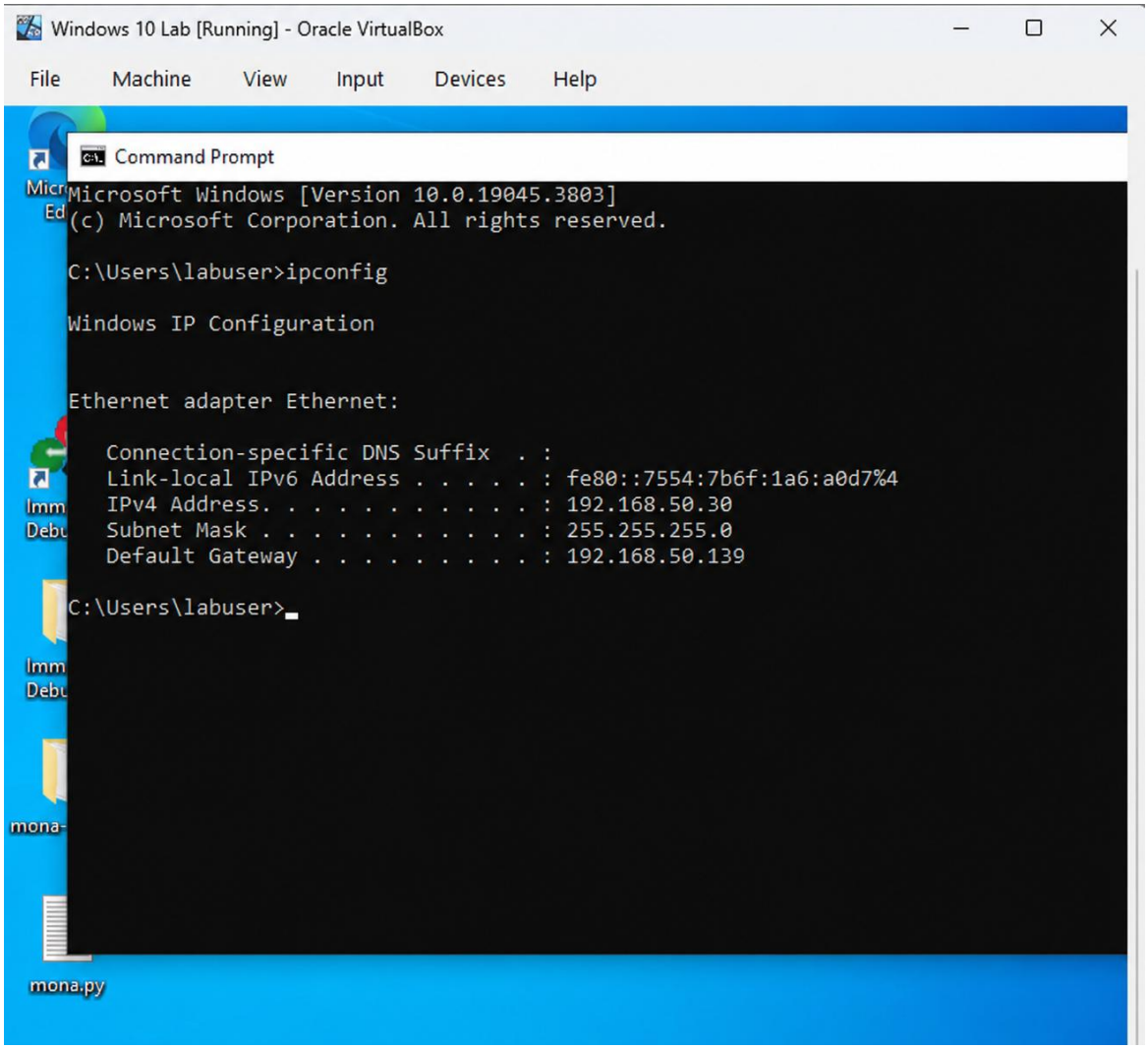
Screenshot 2: Second ifconfig verification on Kali Linux confirming IP address 192.168.50.192.

On the Windows 10 virtual machine, the "ipconfig" command was run from a Command Prompt. The output showed that the Ethernet adapter had the IPv4 address 192.168.50.30 with a subnet mask of 255.255.255.0 and a default gateway of 192.168.50.139.



Screenshot 3: ipconfig output on Windows 10 showing IPv4 address 192.168.50.30.

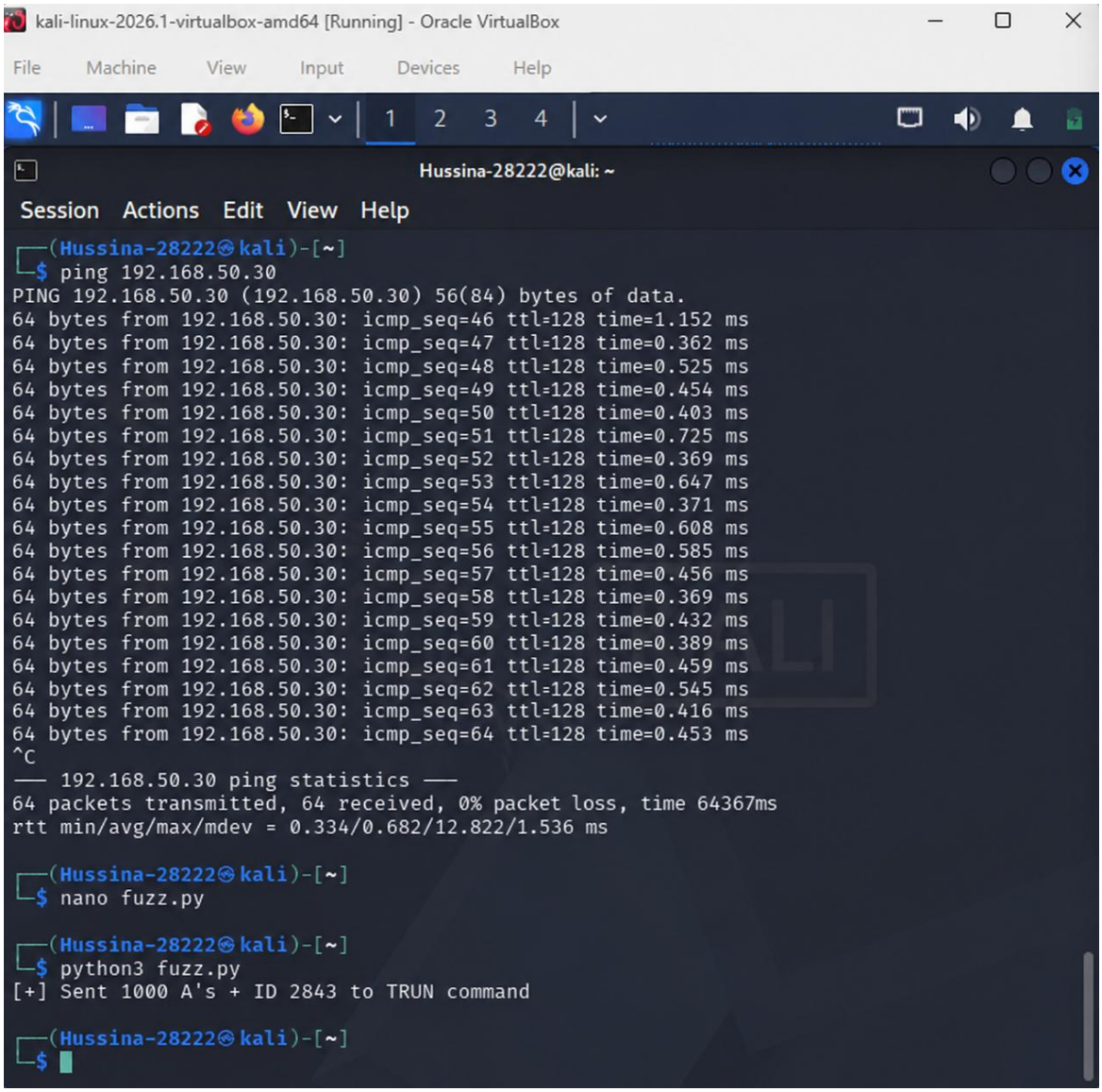
The ipconfig command was run again on Windows 10 to confirm the network configuration was stable.



Screenshot 4: Second ipconfig output on Windows 10 confirming IP address 192.168.50.30.

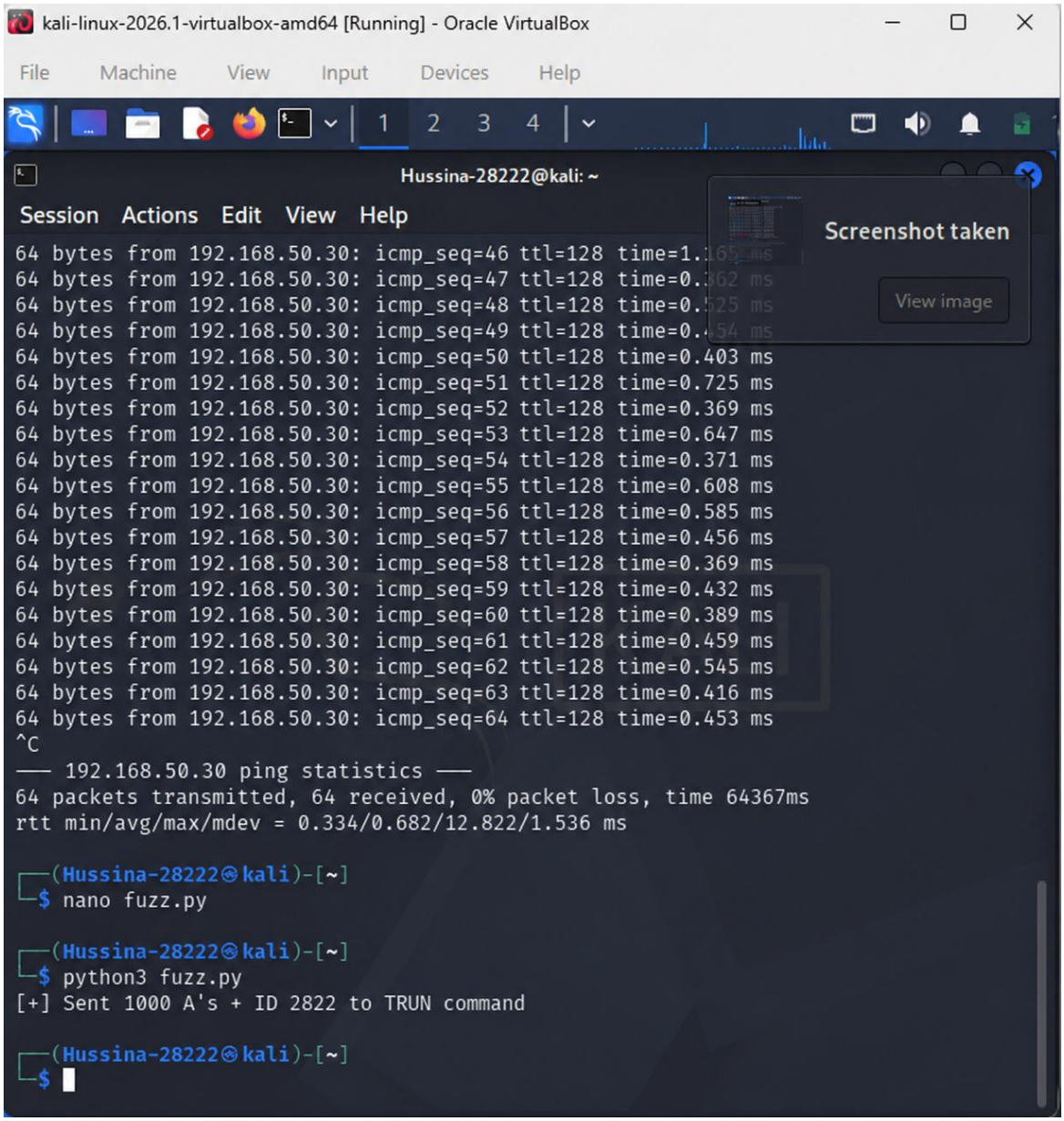
3.2 Testing Network Connectivity with Ping

After confirming the IP addresses, a ping test was run from Kali Linux to Windows 10 to verify that the two machines could reach each other over the network. The command "ping 192.168.50.30" was used. The results showed consistent replies from the Windows 10 machine with low response times, confirming that the network connection between the two virtual machines was working correctly.



Screenshot 5: Kali Linux terminal showing ping command to 192.168.50.30 and successful replies from Windows 10.

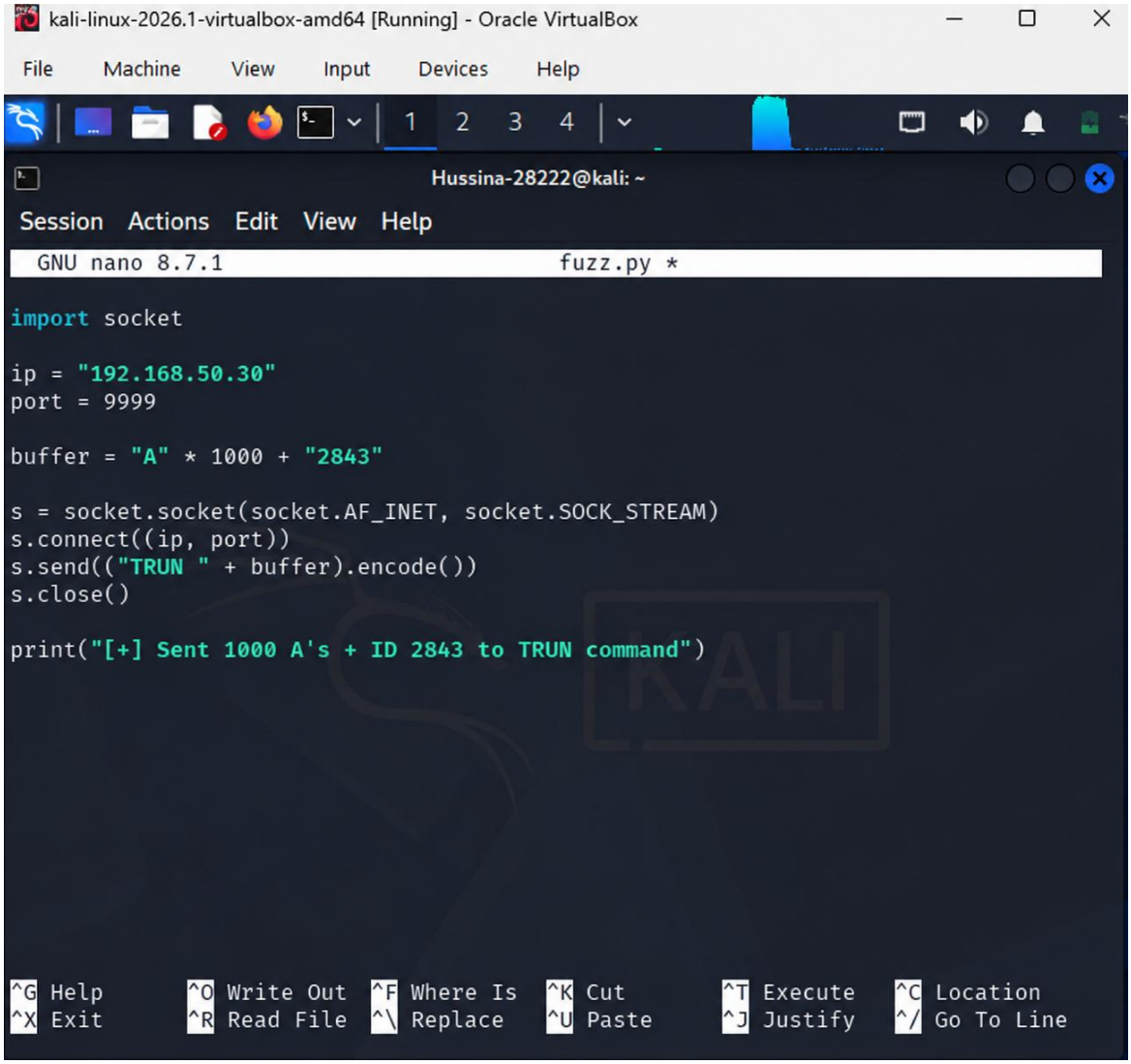
The ping was observed over multiple packets to confirm the connection was stable and not dropping any packets.



Screenshot 6: Continued ping output showing stable and consistent replies from the Windows 10 machine at 192.168.50.30.

3.3 Writing the Fuzzing Script

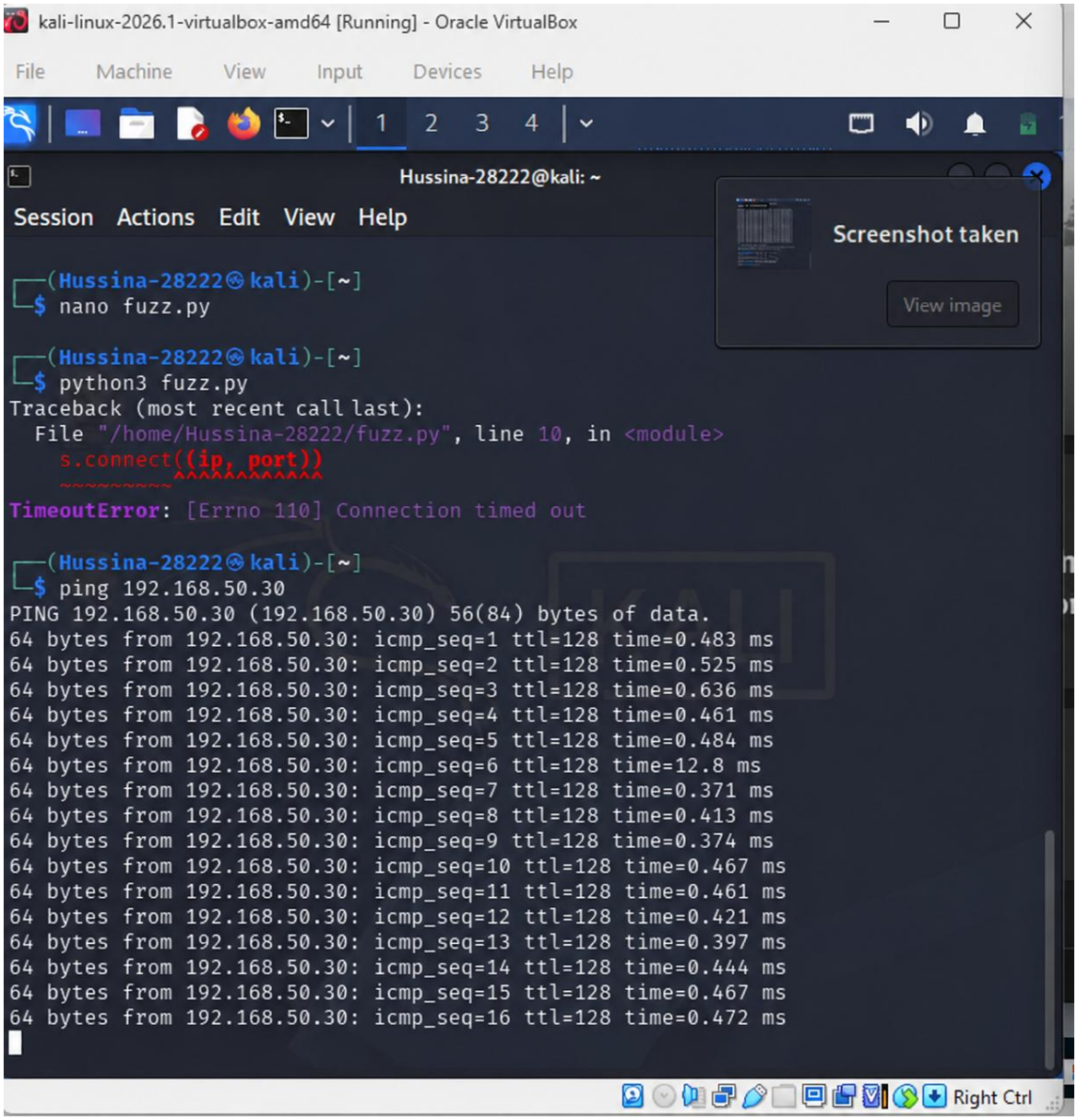
A Python script called fuzz.py was written on Kali Linux using the nano text editor. The script was designed to connect to Vulnserver on port 9999 and send a large payload to the TRUN command to test whether it could be crashed. The script connected to the Windows 10 machine at 192.168.50.30, created a buffer of 1000 A characters followed by the student ID number 28222, and sent the entire string prefixed with the TRUN command. A confirmation message was printed after sending the payload.



Screenshot 7: nano editor showing the fuzz.py script with the TRUN command targeting 192.168.50.30 on port 9999, sending 1000 A characters as the buffer payload.

3.4 Running the Fuzzing Script — First Attempt

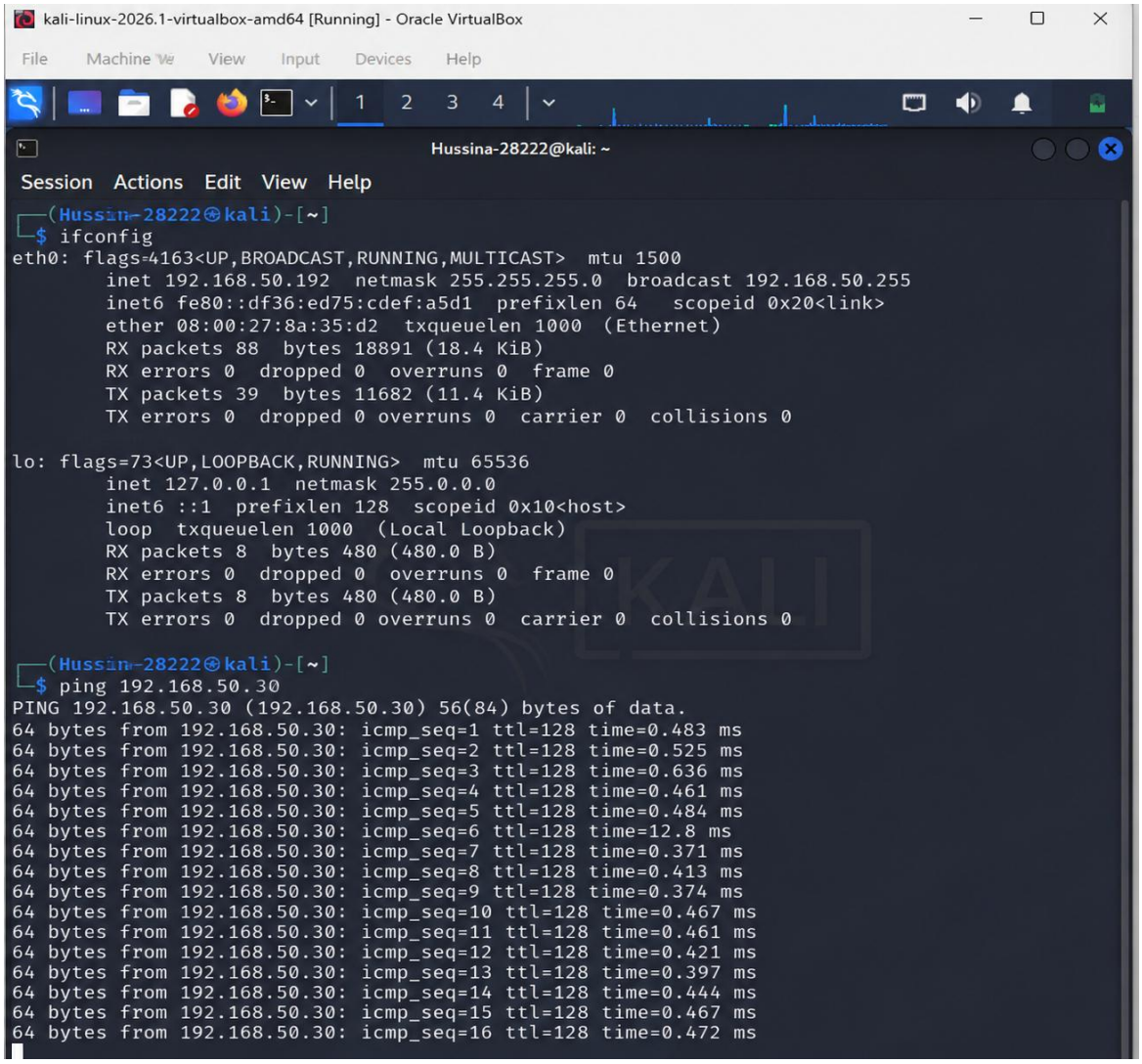
The fuzzing script was run for the first time using the command "python3 fuzz.py". However, the script returned a connection error: TimeoutError [Errno 110] Connection timed out. This error indicated that the script could not reach Vulnserver on the Windows 10 machine. The most likely causes were that Vulnserver had not been started on Windows 10 yet, or that the Windows Firewall was blocking incoming connections on port 9999. A ping test was also visible in the same terminal confirming that basic network reachability was not the issue, as ping replies were still coming through.



Screenshot 8: Kali Linux terminal showing python3 fuzz.py execution resulting in a TimeoutError, indicating Vulnserver was not yet reachable on port 9999.

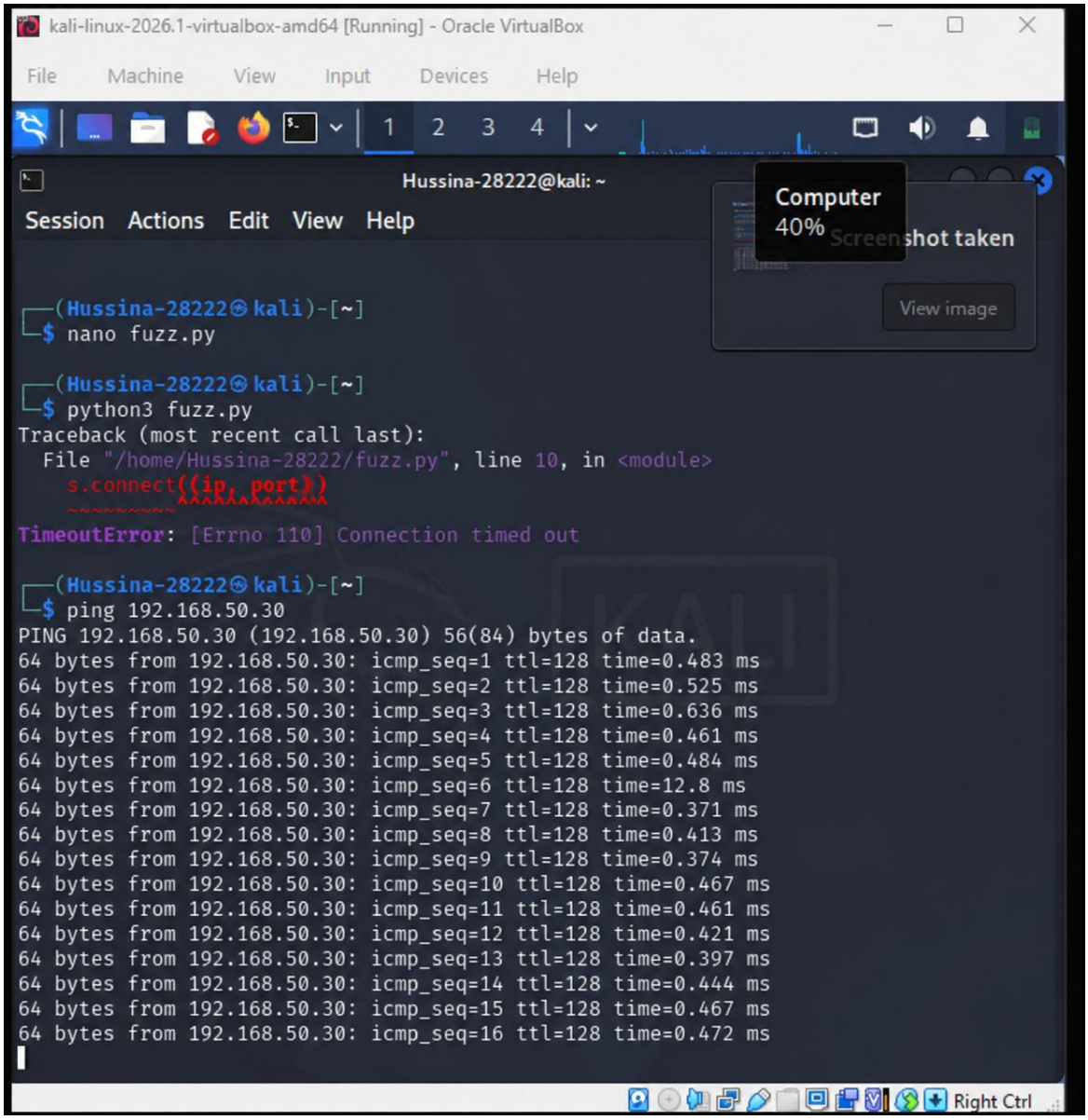
3.5 Rechecking the Network and Re-running the Script

After the first timeout error, the network configuration was verified again by running ifconfig on Kali Linux to confirm the attacker machine IP address was still correct.



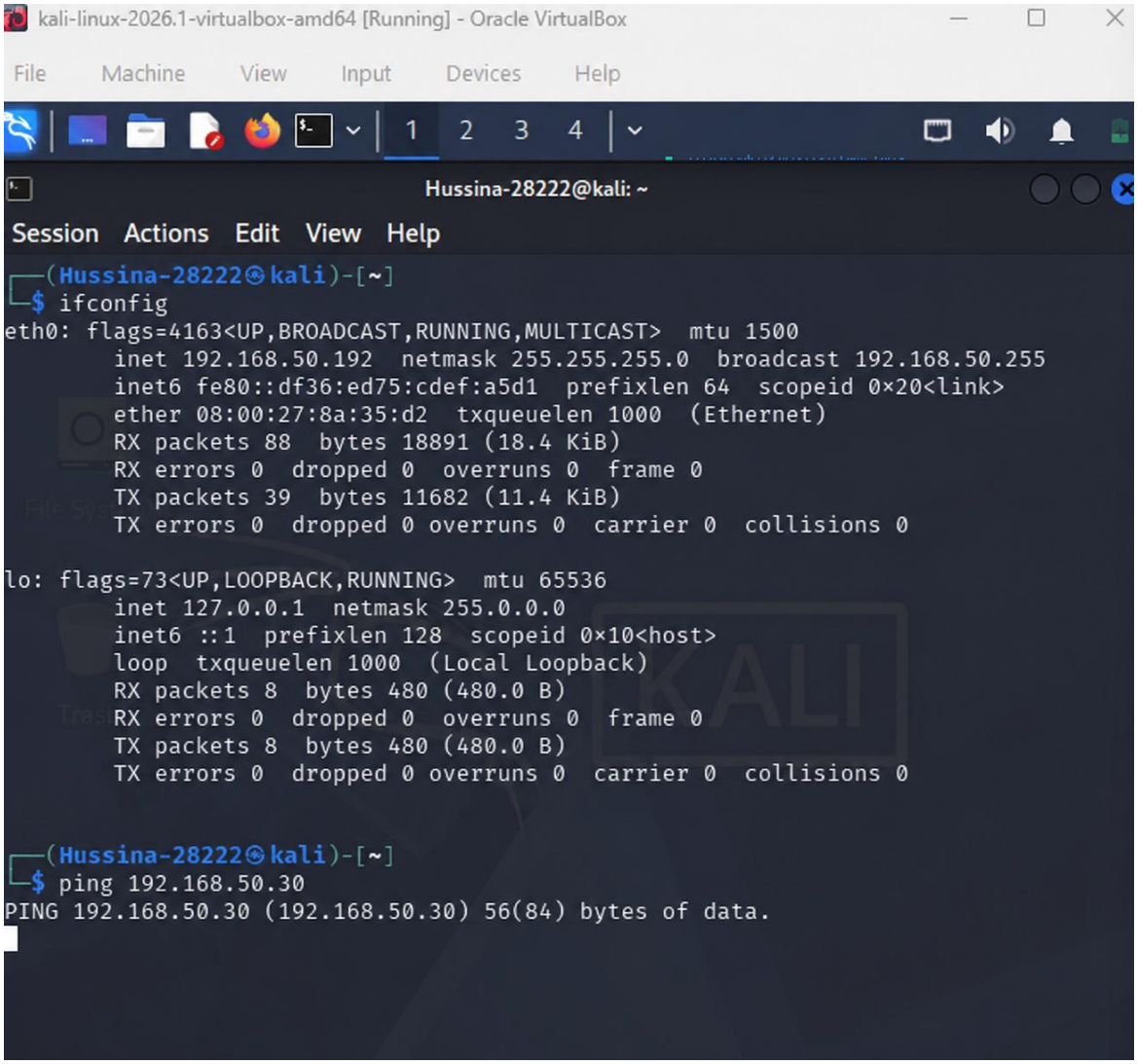
Screenshot 9: ifconfig output confirming Kali Linux IP address 192.168.50.192 remained correct before the second attempt.

The fuzzing script was run a second time after ensuring that Vulnserver was running on the Windows 10 machine. The script again returned a TimeoutError, followed by a ping test that confirmed connectivity to 192.168.50.30 was still working. This confirmed the network was active but Vulnserver needed further configuration, specifically ensuring Windows Defender and the Firewall were both disabled on the victim machine to allow the connection on port 9999.



Screenshot 10: Second run of python3 fuzz.py showing another TimeoutError, followed immediately by a successful ping to 192.168.50.30 confirming basic network connectivity was not the issue.

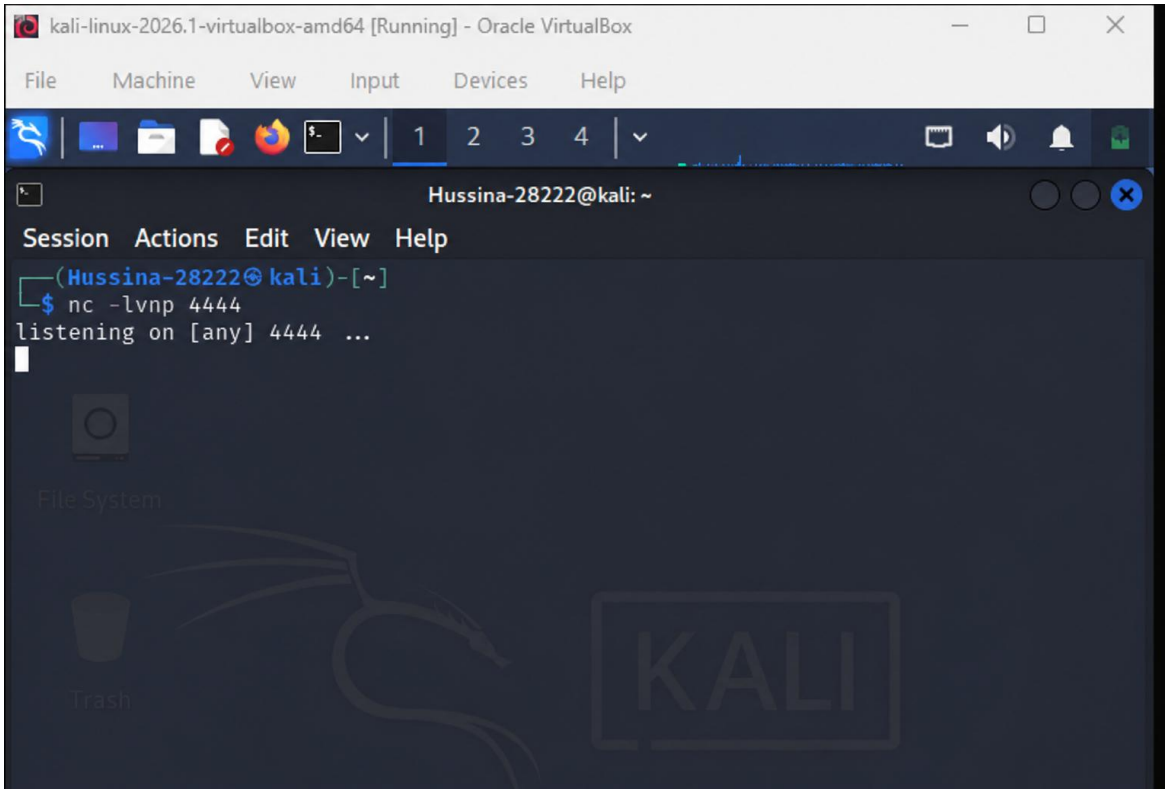
A final ifconfig check was done on Kali Linux after the second attempt to ensure the attacker machine network settings had not changed during the process.



Screenshot 11: Final ifconfig check on Kali Linux confirming IP address 192.168.50.192 after the second fuzzing attempt.

3.6 Setting Up the Reverse Shell Listener

In preparation for receiving the reverse shell once the exploit was successful, a netcat listener was started on Kali Linux using the command "nc -lvnp 4444". This command tells netcat to listen on all network interfaces on port 4444, waiting for an incoming connection. When the exploit payload is delivered and the shellcode executes on Windows 10, the victim machine will connect back to this listener, giving a remote command prompt.

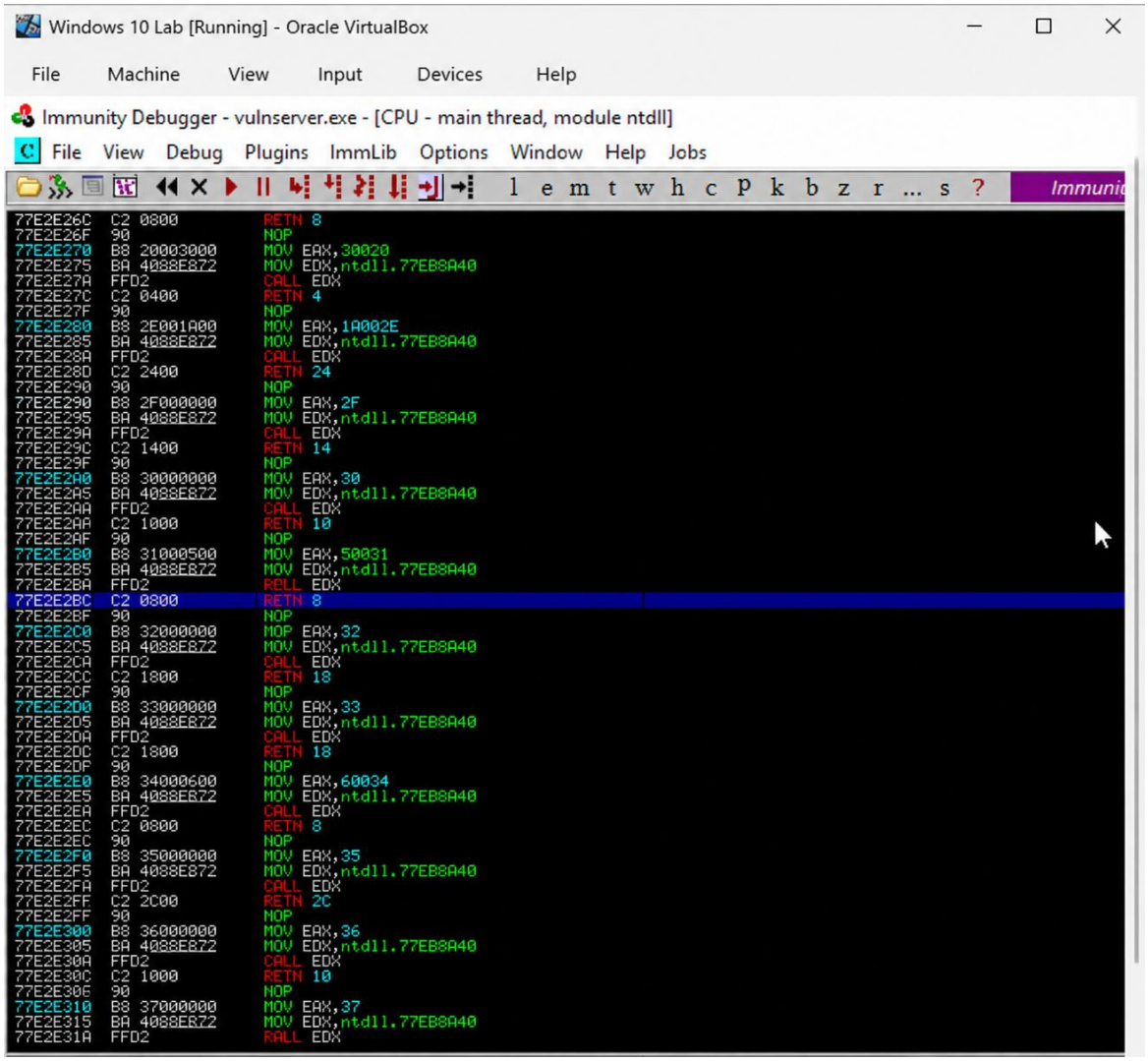


Screenshot 12: Kali Linux terminal showing nc -lvnp 4444 running and listening on port 4444, ready to receive the reverse shell connection from Windows 10.

3.7 Analyzing Vulnserver with Immunity Debugger

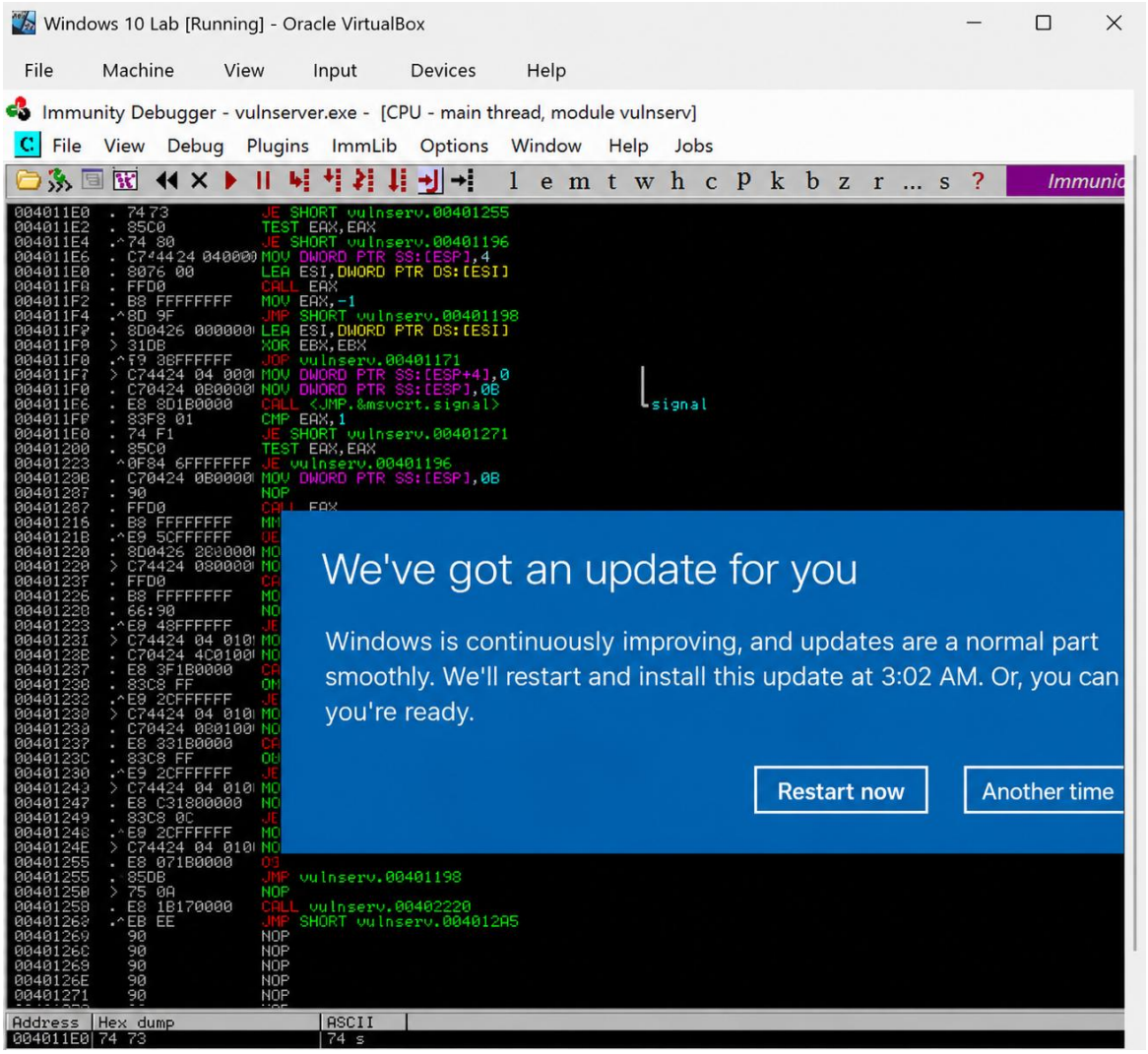
Immunity Debugger was opened on the Windows 10 machine and attached to the running vulnserver.exe process. This allows every aspect of program execution to be monitored in real time. When the fuzzing script sends a large amount of data, the debugger pauses execution at the point of the crash and shows the exact state of the CPU registers and memory at that moment.

The first view in Immunity Debugger showed the ntdll module loaded in memory. The ntdll.dll library is a core Windows system component that handles low-level system calls. Seeing it loaded confirms that Immunity Debugger is actively attached and monitoring the process execution environment, including all modules loaded into the vulnserver process.



Screenshot 13: Immunity Debugger on Windows 10 showing the ntdll module with assembly instructions, confirming the debugger is attached to the vulnserver process and monitoring system-level execution.

The second Immunity Debugger view shows the vulnserver module itself loaded in the CPU main thread. The assembly instructions visible belong to the vulnserver executable, confirming that Immunity Debugger has successfully identified and is analyzing the correct target process. This view is used to observe how execution flows through the vulnerable TRUN command handler when the fuzzing payload is received, and to identify the exact point where the EIP register gets overwritten.



Screenshot 14: Immunity Debugger showing the vulnserver module loaded in the CPU main thread, displaying the assembly instructions of the vulnerable application during analysis.

4. Conclusion

This lab provided hands-on experience with the process of identifying and beginning to exploit a buffer overflow vulnerability in a real but controlled software environment. The steps followed included verifying the IP addresses of both machines, confirming network connectivity through ping tests, writing a Python fuzzing script targeting the TRUN command in Vulnserver, troubleshooting connection errors, setting up a reverse shell listener, and using Immunity Debugger to analyze the running vulnserver process.

The connection timeout errors encountered during fuzzing were a practical reminder that security tools require careful environment setup. Specifically, Windows Defender and the Windows Firewall must be disabled for the vulnerable application to accept connections from the attacking machine. This kind of troubleshooting is a normal part of security testing.

Buffer overflow vulnerabilities like the one in Vulnserver remain a serious category of security flaw because they allow an attacker to take complete control of a running program. As software engineers, understanding how these attacks work is essential for writing safer

code, choosing memory-safe functions, validating input size before writing to buffers, and using compiler protections that make exploitation significantly harder.